

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
Московский авиационный институт
(Национальный исследовательский университет)

Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806

Отчёт по лабораторной работе №1

Дисциплина: «Операционные системы и архитектура компьютера»

Тема: «Анализ профиля системных вызовов программы `ffmpeg`»

Выполнил:
студент группы М8О-206БВ-25
Ширанков М.С.
Проверил:
Ассистент Центра ТОП-ИТ
Ефимов Д.А.

Москва 2026

Содержание

1 Цель работы **2**

2 Ход работы **2**
2.1 Запуск процесса 2
2.2 Настройка окружения 3
2.3 Построение конвейера 3
2.4 Кодирование и запись данных 3
2.5 Завершение процесса 4

3 Получение полного профиля вызовов **4**
3.1 Значение столбцов 5
3.2 Анализ результата 5

4 Разделение профиля по логическим категориям **6**
4.1 Категория: память (`%memory`) 6
4.2 Категория: процессы и потоки (`%process`) 6
4.3 Категория: сигналы (`%signal`) 7
4.4 Категория: файловый ввод-вывод (`%desc`) 7
4.5 Категория: сеть (`%network`) 8

5 Итог работы **8**

1 Цель работы

Построить профиль системных вызовов программы (`ffmpeg`) с помощью утилиты `strace`, получить статистику вызовов и разбить её по логическим подсистемам ядра (память, процессы/потоки, файловый ввод-вывод, сигналы, сеть), не смешивая их между собой.

2 Ход работы

В качестве исследуемой команды используется перекодирование видео `zazu_input.mp4` с явным указанием видеокодека `libx264`:

```
ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4
```

Команда задействует сразу несколько подсистем ОС:

- чтение исходного видео и запись результирующего файла (файловый I/O);
- динамическую загрузку разделяемых библиотек (`libavcodec`, `libx264` и др.) при старте процесса;
- многопоточное кодирование кадров библиотекой `libx264` (создание потоков и их синхронизацию);
- управление памятью под буферы кадров и рабочие структуры кодека.

Ниже разобран построчный (не агрегированный) лог одного запуска `strace -f -s 200 -tt ffmpeg -i zazu_input.mp4 -c:v libx264 out.mp4` по пяти хронологическим фазам — от старта процесса до завершения.

2.1 Запуск процесса

Первая группа вызовов — старт процесса и работа динамического загрузчика:

```
execve("/usr/bin/ffmpeg", ["ffmpeg", "-i", "zazu_input.mp4", ...]) = 0
access("/etc/ld.so.preload", R_OK) = -1 ENOENT
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, ...) = 0
mmap(NULL, 70655, PROT_READ, MAP_PRIVATE, 3, 0) = 0x...
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libavdevice.so.60", ...) = 3
read(3, "\177ELF...", 832) = 832
fstat(3, ...) = 0
mmap(...) mmap(...) mmap(...)
close(3) = 0
```

Эта пятёрка `openat → read → fstat → mmap(×2-4) → close` повторяется для каждой из более чем 100 загружаемых разделяемых библиотек: сначала основные компоненты `ffmpeg` (`libavdevice`, `libavfilter`, `libavformat`, `libavcodec`, `libavutil`, `libx264.so.164`), затем их транзитивные зависимости. После загрузки последней библиотеки главный поток инициализируется:

```
arch_prctl(ARCH_SET_FS, 0x...) = 0
set_tid_address(0x...) = 29257
set_robust_list(0x..., 24) = 0
rseq(0x..., 0x20, 0, 0x53053053) = 0
```

```
mprotect(0x..., 16384, PROT_READ) = 0
mprotect(0x..., 4096, PROT_READ) = 0
...
```

Далее следует ещё по одному `mprotect` на каждый сегмент каждой загруженной библиотеки.

Серия `mprotect(..., PROT_READ)` переводит только что отображённые сегменты библиотек из `writable` (нужного на время линковки) в `read-only` — запись в них больше не требуется.

2.2 Настройка окружения

```
sched_getaffinity(0, 512, [0 1 2 3]) = 16
openat(AT_FDCWD, "zazu_input.mp4", O_RDONLY) = 3
write(2, "Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'zazu_input.mp4':\n", 58) = 58
```

`sched_getaffinity` сообщает ffmpeg доступные ядра CPU (здесь — 4, `[0 1 2 3]`); это число определяет размер будущего пула потоков кодирования. Затем открывается и пробуется (probing) входной контейнер — результат распознавания формата (`mov,mp4,...`) ffmpeg печатает в stderr.

2.3 Построение конвейера

Здесь — главное архитектурное отличие от связки `tar | gzip`: там конвейер строится между отдельными ОС-процессами через `pipe2`+`clone`+`execve` (`sh` запускает `gzip` отдельным процессом). У `ffmpeg` кодек `libx264` слинкован в тот же процесс, поэтому "конвейер"— это не ОС-пайп, а граф **потоков** с общей памятью (`clone3` с флагом `CLONE_VM`), синхронизируемых через `futex` вместо `read`/`write` по дескриптору:

```
openat(AT_FDCWD, "out.mp4", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|...,
        stack=0x..., stack_size=0x7ff400, ...}) = 29304
prctl(PR_SET_NAME, "av:h264:df0") = 0
...
prctl(PR_SET_NAME, "av:h264:df4") = 0
prctl(PR_SET_NAME, "dec0:0:h264") = 0
prctl(PR_SET_NAME, "dmx0:mov,mp4,m4a") = 0
...
prctl(PR_SET_NAME, "mux0:mp4") = 0
```

Из 19 вызовов `clone3` восемь потоков получают явное имя через `prctl(PR_SET_NAME, ...)`, которое прямо видно в трассировке и раскрывает внутреннюю схему конвейера: `dmx0` — поток демультиплексора (разбирает контейнер на потоки), `dec0:0:h264` — поток декодера видеодорожки, `av:h264:df0...df4` — пять потоков покадрового декодирования H.264 (frame-threading), `mux0` — поток мультиплексора, формирующий выходной MP4. Оставшиеся ~11 потоков из 19 — безымянный пул worker-потоков кодера `libx264` (slice-based threading); каждый получает собственный стек ~8 МБ через `mmap(PROT_NONE)`+`mprotect`.

2.4 Кодирование и запись данных

Основная масса вызовов в этой фазе — `futex`: декодер передаёт готовые кадры энкодеру, энкодер раздаёт срезы worker-потокам, каждый из них сигнализирует о готовности через

FUTEX_WAKE/FUTEX_WAIT_BITSET — именно эта активность и даёт те самые 98% времени в системных вызовах из раздела 4.

В выходной файл (дескриптор 4) производится всего **4 вызова** write:

```
write(4, "\0\0\0 ftypisom...free...mdat", 48) = 48
write(4, "...x264 - core 164 ... options: cabac=1 ref=3...", 195063) = 195063
write(4, "\0\2\371\377", 4) = 4
write(4, "\0\0\6\203moov\0\0\0lmvhd...", 1667) = 1667
```

Первый вызов — заголовок контейнера (ftyp/free/заглушка mdat); второй — одним вызовом весь закодированный битовый поток H.264 (195 063 байта, с различимым SEI-префиксом x264 с версией кодека и параметрами кодирования); третий — служебные байты; четвёртый — атом moov (индекс фильма), который MP4-мультиплексор по умолчанию дописывает в конец файла только после того, как все кадры уже записаны.

2.5 Завершение процесса

Когда кодирование окончено, главный поток завершает worker-потоки один за другим — это ядро реализации pthread_join, построенное на futex:

```
futex(0x..., FUTEX_WAKE_PRIVATE, 1) = 1
rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0
madvise(0x..., 8368128, MADV_DONTNEED) = 0
exit(0) = ?
+++ exited with 0 +++
futex(0x..., FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, ..., NULL, ...) = 0
munmap(0x..., 8392704) = 0
```

Главный поток будит очередной рабочий поток (FUTEX_WAKE_PRIVATE); тот блокирует сигналы, освобождает свой стек (madvise(MADV_DONTNEED)) и вызывает exit(0) — завершается только сам поток, а не весь процесс (в отличие от exit_group). Главный поток просыпается из FUTEX_WAIT_BITSET (ожидание на child_tid, выставленного флагом CLONE_CHILD_CLEARTID при создании потока) и освобождает его стек через munmap. Это повторяется по разу на каждый из 19 потоков — прямой функциональный аналог пары wait4/SIGCHLD, которой родительский процесс в сценарии tar | gzip дожидается **дочерних процессов**, только здесь ждут завершения **потоков** того же процесса. После этого главный поток закрывает выходной файл (close(4)) и завершает процесс целиком: exit_group(0).

3 Получение полного профиля вызовов

Флаг -c переводит strace в режим сбора статистики: вместо построчной трассировки каждого вызова выводится статистическая таблица (число вызовов, суммарное и среднее время, количество вызовов, завершившихся ошибкой). Флаг -f обязателен для ffmpeg, так как кодек libx264 создаёт дочерние потоки через clone3() - без -f strace видит только главный поток и теряет всю статистику по кодированию.

```
strace -f -c ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4
```

Результат выполнения (перекодирование тестового видео zazu_input.mp4 на 4-ядерной машине):

```
% time seconds usecs/call calls errors syscall
------ ----------- ----------- --------- --------- ------------------
```

```
 98.19 7.442150 1842 4039 330 futex
  0.63 0.047605 44 1063 mmap
  0.29 0.021859 51 421 mprotect
  0.27 0.020194 196 103 munmap
  0.12 0.009123 37 242 read
  0.12 0.008736 38 228 5 openat
  0.09 0.006535 28 229 fstat
  0.08 0.005932 26 223 close
  0.06 0.004730 32 147 write
  0.04 0.003178 167 19 clone3
  0.03 0.002501 32 77 rt_sigprocmask
  0.03 0.002185 66 33 brk
  0.01 0.000900 45 20 rseq
  0.01 0.000861 45 19 madvise
  0.01 0.000823 41 20 set_robust_list
  0.01 0.000426 30 14 4 prctl
  0.00 0.000195 27 7 lseek
  0.00 0.000163 27 6 pselect6
  0.00 0.000155 31 5 sched_getaffinity
  0.00 0.000129 64 2 2 ioctl
  0.00 0.000116 19 6 rt_sigaction
  0.00 0.000116 38 3 3 access
  ...
------ ----------- ----------- --------- --------- ------------------
100.00 7.579191 1090 6948 348 total
```

3.1 Значение столбцов

- **% time** - доля суммарного времени выполнения системных вызовов, пришедшаяся на данный вызов (не доля времени всей программы - I/O-ожидание и вычисления в пространстве пользователя сюда не входят);
- **seconds** - суммарное время, потраченное ядром на обработку всех вызовов этого типа;
- **usecs/call** - среднее время одного вызова в микросекундах;
- **calls** - сколько раз вызов был выполнен (суммарно по всем потокам, так как использован `-f`);
- **errors** - сколько из этих вызовов завершились с кодом ошибки (`-1` и установленным `errno`);
- **syscall** - имя системного вызова.

3.2 Анализ результата

Основная долю времени, проведённого в системных вызовах, приходится на `futex`. `Libx264` распараллеливает кодирование кадров/срезов (slice-based threading) между рабочими потоками по числу ядер CPU (здесь - 4), а сами потоки создаются вызовом `clone3`. Синхронизация между ними (очередь заданий, условные переменные, мьютексы в реализации NPTL/glibc) реализована поверх futex, поэтому именно он доминирует и по числу вызовов, и по суммарному времени. Остальные вызовы (`mmap`, `mprotect`, `munmap`, `brk`, `madvise`) - это

управление памятью под буферы кадров и стеки потоков, а `read/openat/fstat/close/write` - чтение исходного файла, загрузка динамических библиотек кодеков при старте и запись результата.

4 Разделение профиля по логическим категориям

Готовые классы `strace` (`%memory`, `%process`, `%signal`, `%network`, `%desc`, `%file`, `%ipc`, `%clock`) позволяют собрать статистику только по вызовам одной подсистемы, не смешивая их с остальными. Для каждой категории трассировка запускалась отдельным прогоном, чтобы категории физически не пересекались в выводе:

```
# memory: mmap/munmap/mprotect/brk/madvise
strace -f -c -e trace=%memory -o mem_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# processes/threads: clone3/execve/wait4/exit_group
strace -f -c -e trace=%process -o process_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# signals: rt_sigaction/rt_sigprocmask/tgkill
strace -f -c -e trace=%signal -o signal_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# file descriptors / I/O: read/write/open/close/fstat...
strace -f -c -e trace=%desc -o io_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# network: socket/connect/send*/recv*
strace -f -c -e trace=%network -o net_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4
```

4.1 Категория: память (%memory)

```
% time seconds usecs/call calls errors syscall
------ ----------- ----------- --------- --------- ----------------
 51.55 0.046484 43 1063 mmap
 23.83 0.021487 51 421 mprotect
 20.51 0.018489 179 103 munmap
  3.16 0.002851 150 19 madvise
  0.86 0.000774 23 33 brk
  0.09 0.000079 79 1 mlock
------ ----------- ----------- --------- --------- ----------------
100.00 0.090164 54 1640 total
```

Преобладают `mmap`/`mprotect`/`munmap` - это загрузка динамических библиотек кодеков (`ld.so` отображает каждую `.so` в память и выставляет права страниц) и выделение буферов под кадры или потоковые стеки.

4.2 Категория: процессы и потоки (%process)

```
% time seconds usecs/call calls errors syscall
------ ----------- ----------- --------- --------- ----------------
 82.63 0.002560 134 19 clone3
 17.37 0.000538 538 1 execve
------ ----------- ----------- --------- --------- ----------------
100.00 0.003098 154 20 total
```

Один `execve` - запуск самого процесса `ffmpeg`; 19 вызовов `clone3` - создание потоков пула кодирования `libx264`. При просмотре построчного лога с `-f` по такому количеству потоков часто встречаются строки вида `<unfinished ...>` и `<... resumed>`. `Strace` показывает чередование: если, пока один поток ожидает завершения системного вызова (например, блокирующего `futex` или `read`), планировщик переключается на другой поток и тот тоже входит в syscall, `strace` не может напечатать оба вызова как непрерывные строки - начатый, но ещё не завершившийся вызов помечается `<unfinished ...>`, а когда управление возвращается к его поток и вызов действительно завершается, соответствующая строка получает пометку `<... resumed>` с итоговым результатом.

4.3 Категория: сигналы (%signal)

```
% time seconds usecs/call calls errors syscall
------ ----------- ----------- --------- --------- ----------------
 93.84 0.002454 31 77 rt_sigprocmask
  6.16 0.000161 26 6 rt_sigaction
------ ----------- ----------- --------- --------- ----------------
100.00 0.002615 31 83 total
```

`rt_sigprocmask` вызывается при каждом создании потока (glibc маскирует сигналы на время инициализации нового потока), `rt_sigaction` - установка обработчиков (например, `SIGINT` для корректного завершения по Ctrl+C).

4.4 Категория: файловый ввод-вывод (%desc)

```
% time seconds usecs/call calls errors syscall
------ ----------- ----------- --------- --------- ----------------
 57.66 0.046460 43 1063 mmap
 10.93 0.008807 38 228 5 openat
  9.92 0.007994 32 243 read
  8.11 0.006533 28 229 fstat
  7.93 0.006390 28 223 close
  4.39 0.003540 24 147 write
  0.39 0.000312 44 7 pselect6
  0.26 0.000209 29 7 lseek
------ ----------- ----------- --------- --------- ----------------
100.00 0.080570 37 2158 9 total
```

Здесь видно чтение исходного `zazu_input.mp4`, запись результата и множество пар `openat`/`fstat`/`close` - это динамический загрузчик ищет и открывает разделяемые библиотеки кодеков при старте процесса (5 из 228 `openat` завершились ошибкой `ENOENT` - обычный перебор путей поиска `.so`-файлов).

4.5 Категория: сеть (%network)

Вывод пуст. Поскольку и вход, и выход - локальные файлы, `ffmpeg` в этом сценарии ни разу не обращается к сетевому стеку (ни `socket`, ни `connect`, ни `sendto/recvfrom`).

5 Итог работы

В результате выполнения лабораторной работы были получены практические навыки профилирования реальной многопоточной программы: сбор статистики (`-c`), разделение по классам системных вызовов (`-e trace=%class`), а также разбор сырых трасс для глубокого изучения понимания работы программы.